File Organization in Databases

1. Introduction to File Organization

- A **database** is stored as a collection of files.
- Each **file** is a sequence of **records**.
- A **record** is a sequence of **fields** (attributes).

1.1 Basic Assumptions

- Records are generally **smaller than a disk block**.
- In the simplest design:
  - **Fixed-size records** are assumed.
  - Each file contains only one type of record.
  - Different relations are stored in different files.
- This approach is easiest to implement.

2. Fixed-Length Records

2.1 Storage Layout

- Suppose record size = n bytes.
- Store record i starting from byte position:

$$\text{offset} = n \times (i - 1)$$

- Record access is therefore simple and direct.

2.2 Block Boundary Problem

- Records may cross block boundaries.
- A modification: **do not allow records to cross block boundaries**.
- This improves efficiency by avoiding multi-block reads.

3. Deletion in Fixed-Length Records

When a record is deleted, several approaches can be used:

3.1 Method 1: Shift Records

- Move all records $i + 1, \ldots, n$ to positions $i, \ldots, n - 1$.
- Pros: No gaps, compact layout.
- Cons: Expensive for large files (shifting many records).

3.2 Method 2: Replace with Last Record

- Move the **last record** into the deleted slot.
- Pros: Very efficient (only one copy).
- Cons: Order of records is lost.

3.3 Method 3: Free List Approach

- Do not move records; instead:
  - Maintain a **linked list of free slots**.
  - Insert new records into the free slots.
- Example: If record 3 is deleted, mark it free and link it to other free records.

- Record 3 is deleted.
- Free list updated to point to slot 3.
- Later, record 11 is inserted into slot 3.

4. Summary

- File = sequence of records, record = sequence of fields.
- Fixed-length records simplify access but may cause block-boundary issues.
- Deletion strategies:
  1. Shift subsequent records.
  2. Replace with last record.
  3. Maintain a free list of empty slots.